

MTRN4066 Project Final report

Pranay Harsha Karapakula
Student ID:46846085

July 1, 2026

1 Introduction

The following report explains in detail the process in which a closed-loop controller is created for the given system, the methods used to determine the necessary parameters and how to achieve them are explained. The micro controller used for this project is a teensy 4.0 along with the L298N motor driver as well as a soft potentiometer laid within the beam; the motor used is a NEVEREST orbital 20 gearmotor with an encoder to which the beam is attached.

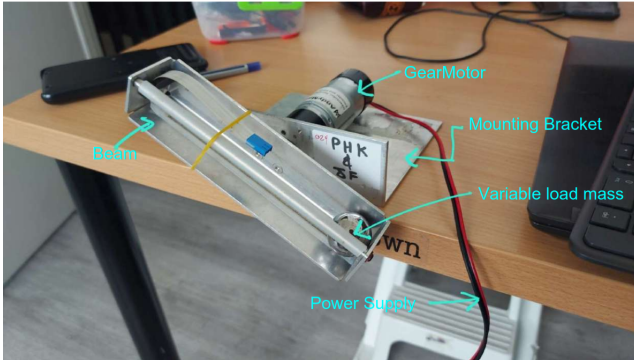


Figure 1: Project setup

2 System parameters

2.1 Mechanical parameters

The following table contains the dimensions of all mechanical components taken into account when calculating the individual moments of inertia. To make calculations easier, the beam side panels, the motor shaft, and the static components of the motor have been ignored.

Table 1: Mechanical parameters

Component	Dimension
motor radius r_r	0.019 m
mass of the motor m_r	0.208 kg
gear reduction GR	19.2 : 1
beam length L_b	0.17 m
beam width W_b	0.035 m
beam mass m_b	0.122 kg
variable load mass m_{LM}	0.08 kg
variable load radius r_{LM}	0.01225 m

When calculating the moment of inertia for the motor,

the shape of the motor was assumed to be a cylinder and taking the gear reduction of the motor into consideration, the formula for inertia is given in equation 1, and by solving it by using the values for the motor in table 1, we get $J_{rotor} = 13.84 \times 10^{-3} \text{ kgm}^2$

$$J_{rotor} = \frac{1}{2} m_r r_r^2 \times 19.2^2 \quad (1)$$

For the beam, we only consider the bottom panel which gives us the moment of inertia equation in equation 2, using the values for the beam in the table we get $J_{beam} = 3.062 \times 10^{-4} \text{ kgm}^2$

$$J_{beam} = \frac{1}{12} m_b (L_b^2 + W_b^2) \quad (2)$$

The variable-load mass for this project is in the shape of a solid sphere and the equation for its inertia is given in equation 3, using the values from the table we get $J_{LM} = 8.04 \times 10^{-6} \text{ kgm}^2$

$$J_{LM} = \frac{2}{5} m_{LM} r_{LM}^2 \quad (3)$$

Here the load mass was calculated as static to make it simple for the system inertia but when system modeling will be done it will be calculated as a variable. The total system inertia can be written as in equation 4 , which gives us $J_{system} = 1.41 \times 10^{-2} \text{ kgm}^2$

$$J_{system} = J_{LM} + J_{beam} + J_{rotor} \quad (4)$$

But when implementing the system the load mass is constantly moving and to account for its movement the inertia can be found using the parallax method which gives us

$$J_{VLM} = \frac{2}{5} m_{LM} r_{LM}^2 + m_{LM} x(t)^2 \quad (5)$$

Where x is the position of the ball along the beam and t is time.

2.2 Electrical characteristics

The table below lists the electrical specifications taken from the datasheet required to model the motor and experimentally determined values are denoted with a *

Table 2: Electrical characteristics

Specification	Value
no load RPM*	306 RPM
stall torque	1.236 Nm
stall current	11.5 A
motor terminal resistance* R	1.24 Ω
motor Inductance*	0.862 mH
motor operating voltage	12 V
encoder operating voltage	3.3 V
encoder resolution	134 PPR
torque constant k_t	0.367 Nm/A
static friction torque τ_{df}	0.065 Nm

To model an accurate motor control system, the torque constant or electronic constant and the speed constant are required, the torque constant K_t was found by recording the rpm values of the motor at different voltages and plotting them in the graph below.

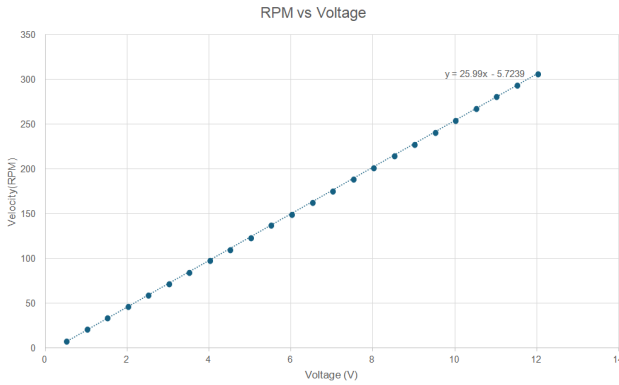


Figure 2: RPM vs Voltage graph

Using the slope of the equation, we get

$$K_t = K_e = \frac{1}{25.99} \cdot \frac{30}{\pi} = 0.367 \frac{Nm}{A} \quad (6)$$

To find the Dynamic friction torque, we need to find the current at which the motor overcomes static friction and by solving for the X intercept we get the applied voltage to overcome static friction which is 0.22 V

$$I_{df} = \frac{V}{R} = \frac{0.22}{1.24} = 0.177 A \quad (7)$$

$$\tau_{df} = K_t I_{df} = 0.065 Nm$$

$$(8) \text{ For modeling the motor response}$$

3 Equations of motion

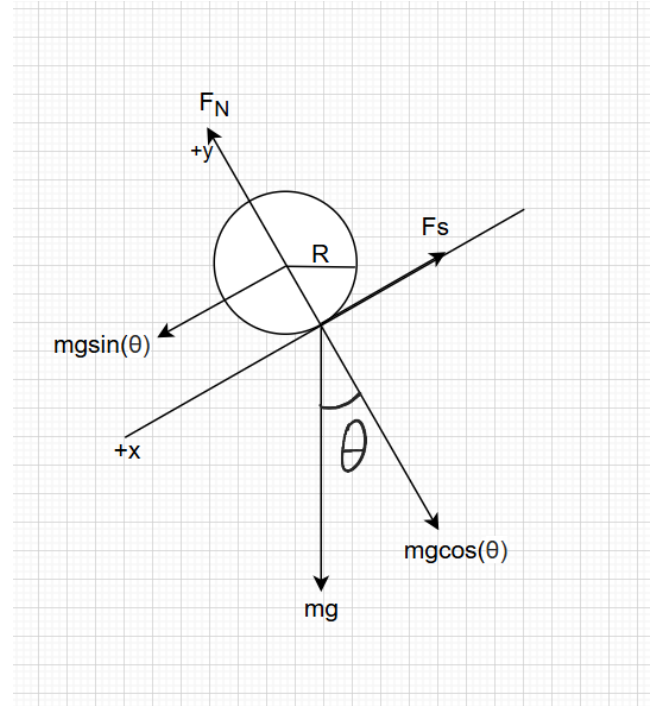


Figure 3: Free Body diagram of the load

The following equations have been derived from the free body diagram of the load on the beam to aid in modeling the system behavior. Assuming the friction acting on the ball is static, we get the sum of forces on the horizontal as

$$\sum F_x = m\ddot{x} \quad (9)$$

and the sum of rotational forces

$$\sum \tau = J\ddot{\theta} \quad (10)$$

which gives us

$$f_s = mgsin\theta - m\ddot{x} \quad (11)$$

where

$$f_s = J_{vlm}\ddot{x} \quad (12)$$

substituting eq 10 into eq 11 we get,

$$gsin\theta - \ddot{x} = \frac{2}{5}\ddot{x} \quad (13)$$

or we can rewrite it as

$$\ddot{x} = \frac{5}{7}gsin\theta \quad (14)$$

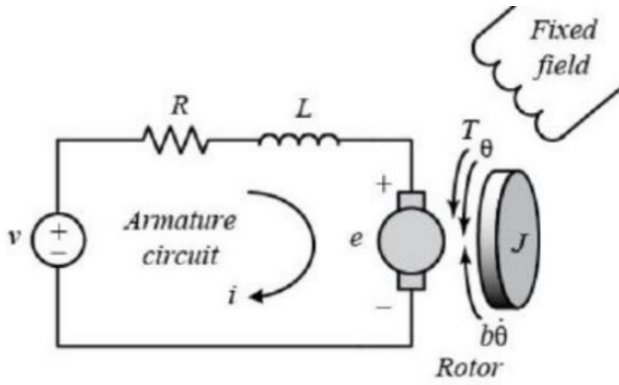


Figure 4: Equivalent circuit of motor

using KVL around the circuit

$$V = IR + L \frac{di}{dt} + k_b \omega \quad (15)$$

or

$$\dot{\theta} = \frac{V - IR + L \frac{di}{dt}}{k_b} \quad (16)$$

From rearranging eq 9 we can get

$$\ddot{\theta} = \frac{\tau_m - \tau_{df}}{J_{system}} \quad (17)$$

or

$$\ddot{\theta} = \frac{Ik_t - \tau_{df}}{J_{total}} \quad (18)$$

And substituting eq 15 into eq 18 we get an equation that relates acceleration to torque while ignoring the variable load mass in the system

$$\ddot{\theta} = \frac{\frac{V - k_b \dot{\theta} - L \frac{di}{dt}}{R} k_t - \tau_{df}}{J_{total}} \quad (19)$$

4 System modelling using Euler's method

The system we are working with is a discrete system by virtue of using a teensyduino and Euler's method allows us to model complex systems with multiple components with any timestep we want. The following equations were used to integrate the equations we derived in section 3, this method allows us to change the position and velocity at each time step based on the current and derived values

$$v_{i+1} = v_i + a \cdot \Delta t \quad (20)$$

$$x_{i+1} = x_i + v_i \cdot \Delta t \quad (21)$$

where Δt is the chosen timestep and v_{i+1} and x_{i+1} are the next values based on the current and derived values at a certain timestep. These equations were used in Matlab to allow us to properly simulate the open-loop response seen in the section below

5 Open loop response

To gain the system response of the load in real life, we tilted the beam at an angle and dropped the ball while recording the change in position through the flexible potentiometer within the beam. The angle of the beam was measured through the encoder within the motor. Once we got the data points, we plotted the real-life response against the simulated response, which gave us this graph.

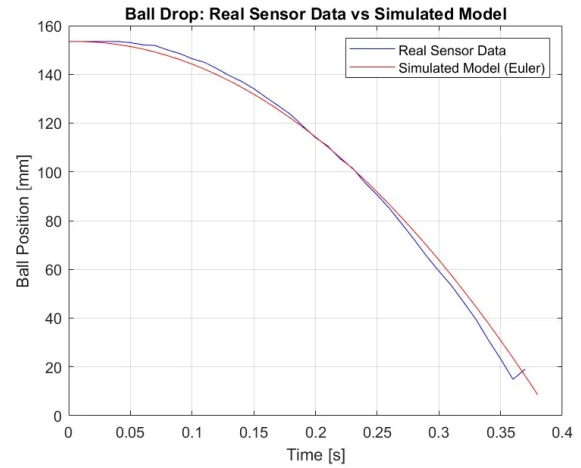


Figure 5: Real and simulated open-loop response of load

There is a slight difference in the two models which could be due to errors like the smoothness of the ball causing it to slip which can give misreadings and when it reaches the end it bounces back up causing the slight spike in the graph, but these errors are very small so we can ignore them. For the motor response, we used the encoder to record the data of the motor from no speed to when it reached its nominal speed with the beam attached and plotted it against the simulated response which gave us this graph.

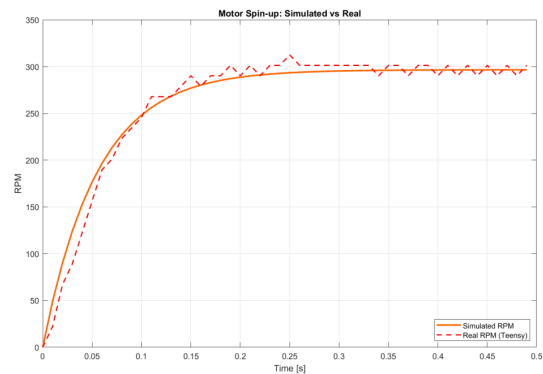


Figure 6: Open-loop motor response, simulated and real

The real response does not look as smooth as the simulated response, and that could be due to double counting or misreading of the encoder.

6 Closed-loop modeling

The previous section dealt with the system working without the feedback from the position sensor, which is why to control the position of the ball a feedback loop equation was designed using the position sensor to control the voltage output to be added to the system.

6.1 Closed-loop equation

The Voltage input can be expressed as the gain multiplied by the errors between the desired output and the actual output of the system in Equation 22.

$$V(t) = -k_p e - k_d v(t) - k_a(t) \quad (22)$$

Where e, v and a are the errors of position, velocity and acceleration of the variable load mass respectively. The gains for each term are k_p, k_d , and k_a respectively. The integral term was omitted since the encoder was not used for feedback on accurate integration of the beam angle and instead an acceleration term was used to improve the damping and response time of the system

6.2 Closed-loop response

The values for the gains k_p, k_d , and k_a are 153 V/m, 28.2 V/m, and 1.84 V/m respectively for the variable load mass to reach 0mm from 75mm .

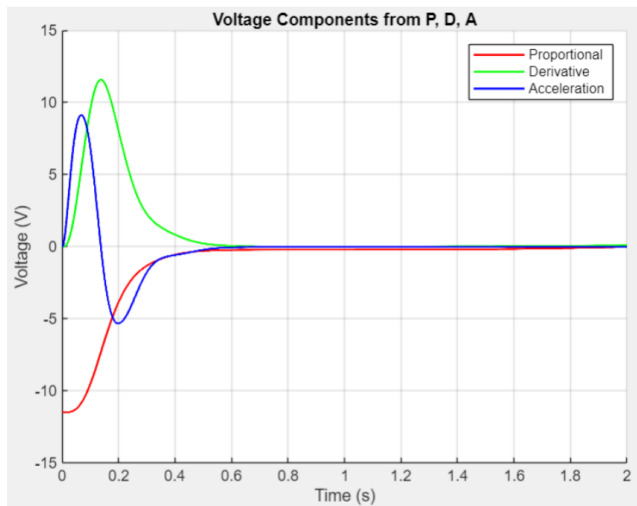


Figure 7: Closed-loop response

observing Figure 7 we can see that the proportional gain is countered by the velocity and acceleration gain. The addition of the beam angle gain and beam angle velocity gain adds stability to the system, which allows us to have a low settling time at 0.5 s with virtually no overshoot, which is extremely good for a system to have and a sse of 1.73% .

6.3 Variable load mass response

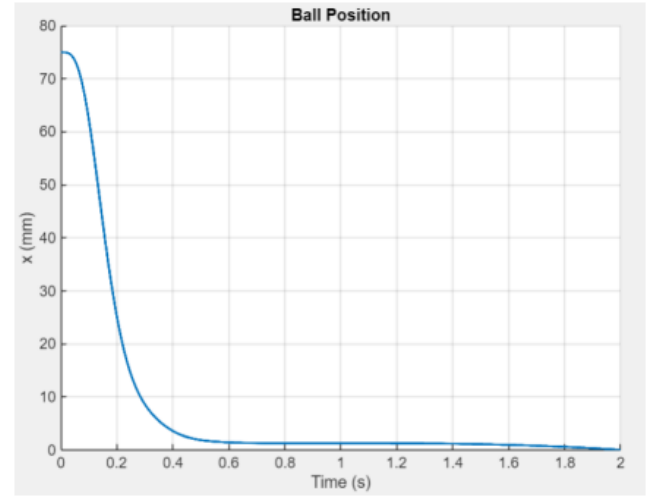


Figure 8: Variable load mass response in closed-loop

Figure 8 shows the response of the variable load mass when the closed-loop system has been integrated into the solution, However after 3s minor oscillations appear from underdamped terms by the small control variations. The limited amount of precision may lead to small voltage fluctuations that can excite the system and the acceleration gain can also amplify the disturbances when the variable load mass is trying to settle. Any oscillations after 3s were excluded as the occurred after the settling window and would most likely be damped in the implementation of the real system.

7 Real system implementation

7.1 Control implementation for teensy

The teensy calculates the position by taking the position through the soft potentiometer by digitizing the analog signal which cause it to have noise, which is then increased when the derivatives of velocity and acceleration are taken. To get smooth derivatives without relying on an encoder a second order polynomial regression was applied with a rolling buffer of 15 time fitted samples per cycle using a curve fitting library.

7.2 Comparison of simulated and real response

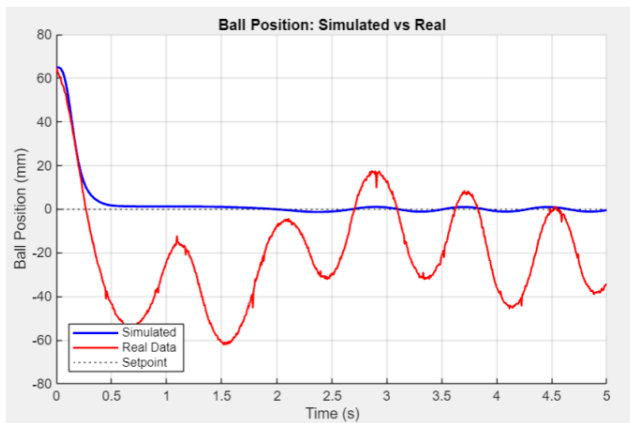


Figure 9: simulated vs real ball position

The simulated response in figure 8, shows a very smooth graph with negligible overshoot, low settling time and steady state error. On the other hand, the real system displays an overshoot up to 70mm, sse of 40% and undefined settling time due to underdamped oscillations. Some of the issues identified are the High proportional gain which increased the swinging of the beam, a lag with the curve fit due to large buffer windows that reduces responsiveness, the output was clamped to reduce over swinging limiting the power sent to the motor and the dead zone logic made the controller unresponsive near the desired point.

7.3 Neural network implementation

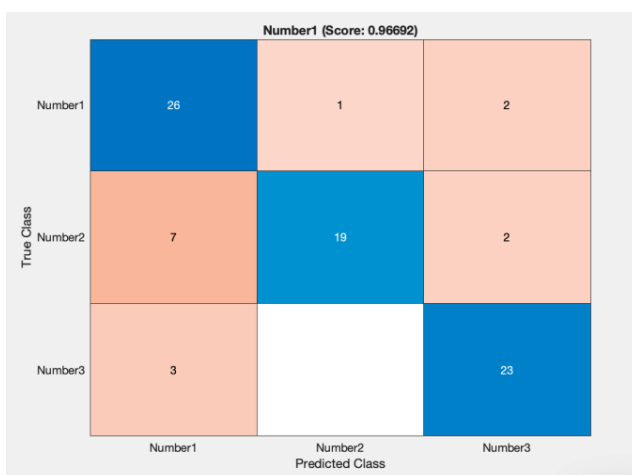


Figure 10: Neural net confusion matrix

A neural network was implemented by using MATLAB's deep learning toolbox to recognize and classify hand gestures in real time. The network was trained to recognize hand gestures that represented one, two, and three. Figure 10 shows the confusion matrix, displaying reasonably low misclassification. Accuracy can be improved by increasing the data set and filtering the input

images. Each gesture sent a specific command string over the serial port to the teensy and would change the desired setpoint which was predefined depending on what gesture the network recognized. This was validated by using LED's to simulate a setpoint connected to a specific gesture and having them light up when said gesture was displayed.

8 Conclusion

The simulation of ball and beam system was successful, displaying very good performance metrics but the real implementation by various handling, tuning and latency issues. Further improvements on this system should be focused on optimizing the curve fit algorithm, reducing lag and better tuning of gains.

9 appendix

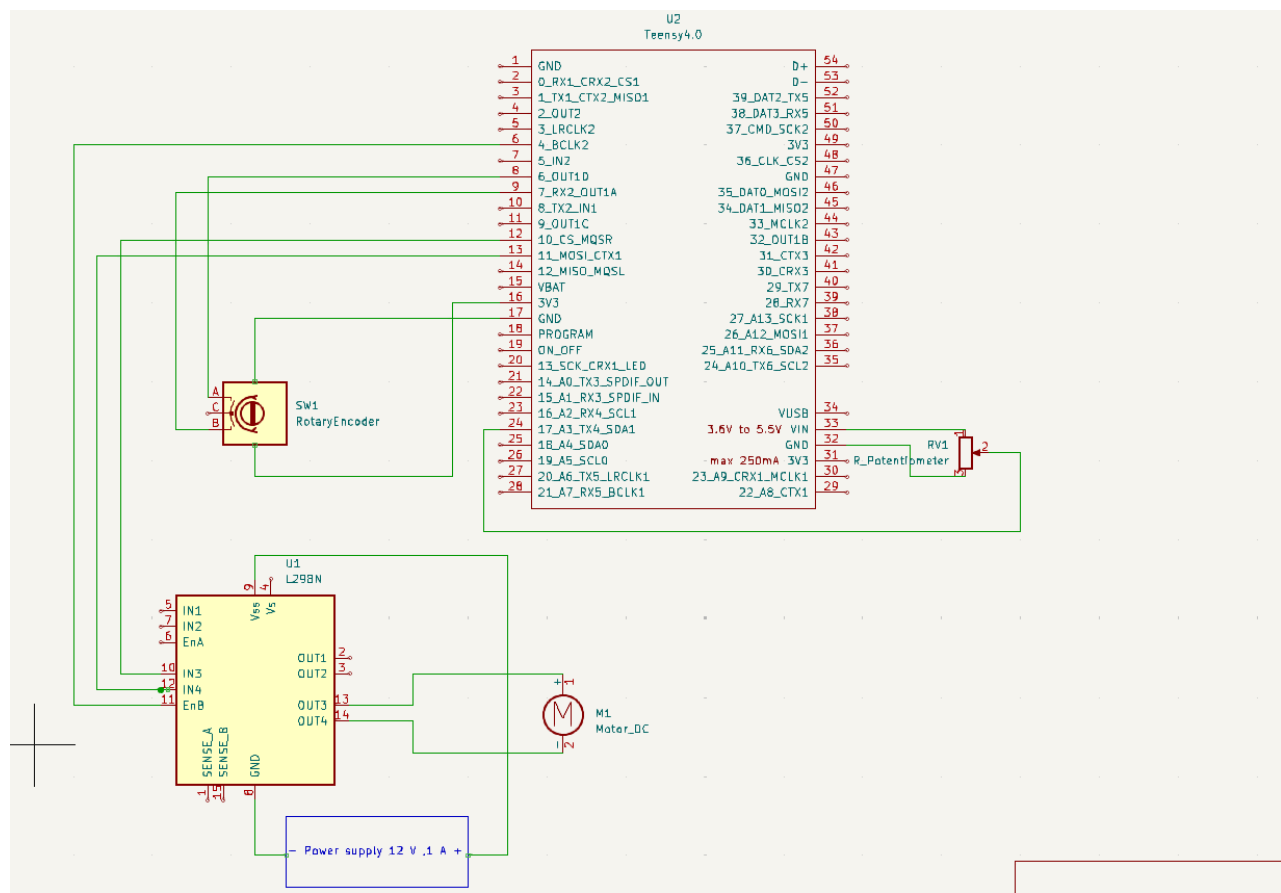


Figure 11: Circuit diagram for the setup